AFRILANG Stdlib

std/c/comprle

Français. Bibliothèque AFRILANG ‘std/c/comprle‘ (niveau complex, catégorie complex). Elle expose 7 fonction(s) : rleEncode, rleDecode, rleRatio, runLength, compress, decompress, isCompressible.
‘import ”std/c/comprle”‘ · ‘use comprle‘
- ‘rleEncode(data list number) → list number‘ - ‘rleDecode(encoded list number) → list number‘ - ‘rleRatio(data list number) → number‘ - ‘runLength(data list number) → number‘ - ‘compress(data list number) → list number‘ - ‘decompress(encoded list number) → list number‘ - ‘isCompressible(data list number) → bool‘ **English.** AFRILANG library ‘std/c/comprle‘ (tier complex, category complex). Exports 7 function(s): rleEncode, rleDecode, rleRatio, runLength, compress, decompress, isCompressible.
‘import ”std/c/comprle”‘ · ‘use comprle‘
- ‘rleEncode(data list number) → list number‘ - ‘rleDecode(encoded list number) → list number‘ - ‘rleRatio(data list number) → number‘ - ‘runLength(data list number) → number‘ - ‘compress(data list number) → list number‘ - ‘decompress(encoded list number) → list number‘ - ‘isCompressible(data list number) → bool‘

Catégorie / Category	Modules complex (c/)	
Niveau / Tier	complex	June 16, 2026
Fonctions / Functions	7	

Contents

Français

1. Vue d’ensemble

Bibliothèque AFRILANG ‘std/c/comprle‘ (niveau complex, catégorie complex). Elle expose 7 fonction(s) : rleEncode, rleDecode, rleRatio, runLength, compress, decompress, isCompressible.

‘import ”std/c/comprle”‘ · ‘use comprle‘

```
- `rleEncode(data list number) → list number`
- `rleDecode(encoded list number) → list number`
- `rleRatio(data list number) → number`
- `runLength(data list number) → number`
- `compress(data list number) → list number`
- `decompress(encoded list number) → list number`
- `isCompressible(data list number) → bool`
```

2. Installation & import

Ajoutez le module à votre programme :

```
import "std/c/comprle"
use comprle
```

```
Niveau : complex · Catégorie : Modules complex (c/)
Domaines avancés — graphes, NLP, réseaux complexes.
```

3. Référence API

```
•
 rleEncode(data list number) → list•
 rleDecode(encoded list number) → list•
 rleRatio(data list number) → number•
 runLength(data list number) → number•
 compress(data list number) → list•
 decompress(encoded list number) → list•
 isCompressible(data list number) → bool
```

4. Exemples d’utilisation

```
import "std/c/comprle"
use comprle
```

```
create result = rleEncode(/* args */)
say result
```

```
create result = rleDecode(/* args */)
say result
```

```
create result = rleRatio(/* args */)
say result
```

```
create result = runLength(/* args */)
say result
```

```
create result = compress(/* args */)
say result
```

```
create result = decompress(/* args */)
say result
```

5. Bonnes pratiques

```
•
 Préférez `import "std/c/comprle"` en tête de fichier.•
 Lancez `afrilang check` avant de livrer.•
 Combinez avec les modules stdlib de la même catégorie.•
 Voir /stdlib/ pour le source live et les démos playground.•
 Signalez les problèmes sur GitHub si une export manque.
```

6. Annexe — source

module comprle

```
function rleEncode(data list number) returns list number
end
```

```
function rleDecode(encoded list number) returns list number
end
```

```
function rleRatio(data list number) returns number
end
```

```
function runLength(data list number) returns number
end
```

```
function compress(data list number) returns list number
end
```

```
function decompress(encoded list number) returns list number
end
```

```
function isCompressible(data list number) returns bool
end
end
```

English

1. Overview

AFRILANG library ‘std/c/comprle‘ (tier complex, category complex). Exports 7 function(s): rleEncode, rleDecode, rleRatio, runLength, compress, decompress, isCompressible.

‘import ”std/c/comprle”‘ · ‘use comprle‘

```
- `rleEncode(data list number) → list number`
- `rleDecode(encoded list number) → list number`
- `rleRatio(data list number) → number`
- `runLength(data list number) → number`
- `compress(data list number) → list number`
- `decompress(encoded list number) → list number`
- `isCompressible(data list number) → bool`
```

2. Installation & import

Add the module to your program:

```
import "std/c/comprle"
use comprle
```

```
Tier: complex · Category: Complex modules (c/)
Advanced domains — graphs, NLP, complex networks.
```

3. API reference

```
•
 rleEncode(data list number) → list•
 rleDecode(encoded list number) → list•
 rleRatio(data list number) → number•
 runLength(data list number) → number•
 compress(data list number) → list•
 decompress(encoded list number) → list•
 isCompressible(data list number) → bool
```

4. Usage examples

```
import "std/c/comprle"
use comprle
```

```
create result = rleEncode(/* args */)
say result
```

```
create result = rleDecode(/* args */)
say result
```

```
create result = rleRatio(/* args */)
say result
```

```
create result = runLength(/* args */)
say result
```

```
create result = compress(/* args */)
say result
```

```
create result = decompress(/* args */)
say result
```

5. Best practices

```
•
 Prefer `import "std/c/comprle"` at the top of your file.•
 Run `afrilang check` before shipping.•
 Combine with related stdlib modules from the same category.•
 See /stdlib/ for the live source and playground demos.•
 Report issues on GitHub if an export is missing or undocumented.
```

6. Appendix — source

module comprle

```
function rleEncode(data list number) returns list number
end
```

```
function rleDecode(encoded list number) returns list number
end
```

```
function rleRatio(data list number) returns number
end
```

```
function runLength(data list number) returns number
end
```

```
function compress(data list number) returns list number
end
```

```
function decompress(encoded list number) returns list number
end
```

```
function isCompressible(data list number) returns bool
end
end
```

Référence rapide / Quick reference

- Import : `import "std/c/comprle"`
- Vérification : `afrilang check`
- Documentation : https://afrilang.dev/stdlib/std/c/comprle/

Source (extrait)

```
module comprle

function rleEncode(data list number) returns list number
end

function rleDecode(encoded list number) returns list number
end

function rleRatio(data list number) returns number
end

function runLength(data list number) returns number
end

function compress(data list number) returns list number
end

function decompress(encoded list number) returns list number
end

function isCompressible(data list number) returns bool
end
end
```